

Information distillation from physical simulation

Damian Gierek

07.02.2025

In this paper, I use the method of unsupervised learning of equations of motion for objects in raw and optionally distorted unlabelled synthetic video, presented in this article [1], for the case of videos of bodies moving in gravitational field to obtain the trajectories of the bodies. To obtain this, I trained an autoencoder that maps each video frame into a low-dimensional latent space that allows reproducing the original image. The desired 2D Cartesian trajectory is obtained via principal component analysis of vectors from the latent space. The paper presents preliminary results.

1 Motivation and methodology

First, let us clarify why this approach is interesting. By obtaining the trajectory of a moving body from a video, we can guess the formula that governs the movement of the body using symbolic regression. An example of an algorithm that allows it is "AI Feynman" [2]. This approach was analogous to the work of Johannes Kepler, who discovered that Mars' orbit is an ellipse after analysing the data of its orbit [3].

The method is based on training a specific architecture consisting of an autoencoder and an evolution operator performed on a set of snapshots of a simulation of a body moving in a gravity field.

2 Code library

The whole code I used for the project is publicly available here: https://github.com/dgierek/physics_guesser.

The library contains the module "data_generator.py" that is responsible for generating simulation data. The module uses another module called "rocket_simulation.py" that stores the analytical solutions used in simulation generation. Simulations are stored in the file "simulation_frames".

The module called "autoencoder.py" contains an implementation of used neural network architecture. Whilst the module "datasets.py" contains data loaders that inherit from torch.utils.data.Dataset and load the simulation data during training sessions. The training loop is implemented in the module called "trainer.py".

Finally, in the "evaluator.py" module I implemented the method for calculating the loss function. The "evaluation.py" module stores the implementation of principal component analysis and visualisation of images and trajectories reconstructed by the neural network.

2.1 Simulation of a body moving in a gravity field

The first problem to solve for the project was to create a set of data with which the neural network will be trained. In the beginning I decided to try to simulate the movement in a gravity or magnetic field, due to harmonic oscillator force and the movement of a free body using numerical

integration of differential equations of motion via the Verlet method. Then, I realised it is not necessary as using the analytical solutions is sufficient. Below I show the equations of motion for each type, the analytical solutions I used for simulation and an example of a generated trajectory:

- gravity:

$$\begin{aligned}\vec{F}_g &= m\vec{g} \\ x(t) &= \frac{g_x \cdot t^2}{2} + V_{0,x}t + x_0 \\ y(t) &= \frac{g_y \cdot t^2}{2} + V_{0,y}t + y_0,\end{aligned}$$

where $\vec{F}_g$ is the force of gravity, m is mass of the body (in every calculation $m = 1$ was assumed), $\vec{g}$ is the gravity acceleration vector and its x and y components: g_x , g_y , while $x(t)$, $y(t)$ is the x and y component of the position of the body at a given time t . x_0 and y_0 describe the initial position. $V_{0,x}$ and $V_{0,y}$ describe the initial velocity. An example trajectory was shown on the figure 1.

- magnetic field:

$$\begin{aligned}m\ddot{x} &= q \cdot \dot{y} \cdot B_z \\ m\ddot{y} &= -q \cdot \dot{x} \cdot B_z \\ x(t) &= x_0 + \frac{1}{\omega}(v_{0,x} \sin(\omega t) + v_{0,y}(1 - \cos(\omega t))) \\ y(t) &= y_0 + \frac{1}{\omega}(v_{0,x}(\cos(\omega t) - 1) + v_{0,y} \sin(\omega t)),\end{aligned}$$

where double dots represent the second time derivative and single dot represent the first time derivative. q is the electric charge of a body, B_z is the z component of the magnetic field vector (in the 2D case only this component influences the movement) and $\omega = \frac{qB_z}{m}$. An example trajectory was shown on the figure 2.

- harmonic oscillator:

$$\begin{aligned}m\ddot{x} &= -k_x(x - r_{0,x}) \\ m\ddot{y} &= -k_y(y - r_{0,y}) \\ x(t) &= (x_0 - r_{0,x}) \cos(\omega_x t) + \frac{v_{0,x}}{\omega_x} \sin(\omega_x t) + r_{0,x} \\ y(t) &= (y_0 - r_{0,y}) \cos(\omega_y t) + \frac{v_{0,y}}{\omega_y} \sin(\omega_y t) + r_{0,y},\end{aligned}$$

where k_x , k_y are spring constants in directions x and y , $r_{0,x}$ and $r_{0,y}$ are the coordinates of the equilibrium position of the harmonic oscillator (at this point the force acting on the body is $\vec{0}$). $\omega_x = \sqrt{\frac{k_x}{m}}$ and $\omega_y = \sqrt{\frac{k_y}{m}}$. An example trajectory was shown on the figure 3.

- free body movement:

$$\begin{aligned}\vec{F}_f &= \vec{0} \\ x(t) &= x_0 + v_{0,x}t \\ y(t) &= y_0 + v_{0,y}t.\end{aligned}$$

An example trajectory was shown on the figure 4.

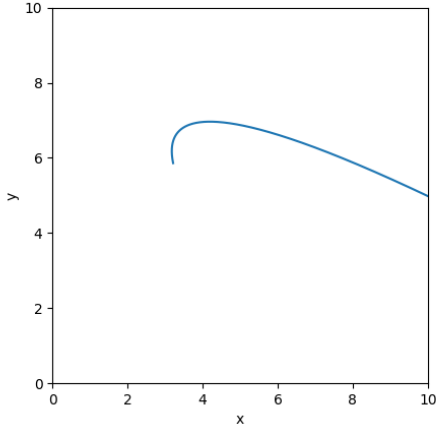


Figure 1: Example trajectory of a body moving in a gravity field. Parameters: $x_0 = 3,22$, $y_0 = 5,86$, $V_{0,x} = -0,17$, $V_{0,y} = 0,82$, $g_x = 0,40$, $g_y = -0,31$.

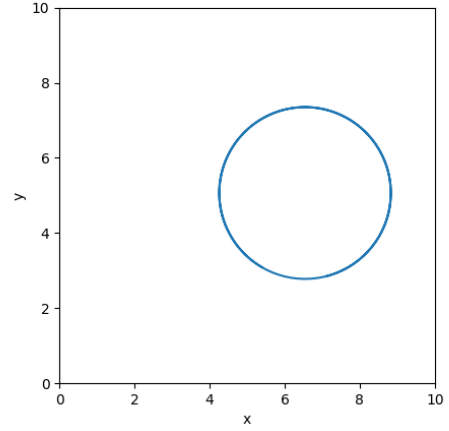


Figure 2: Example trajectory of a body moving in a magnetic field. Parameters: $x_0 = 5,40$, $y_0 = 3,09$, $V_{0,x} = -0,78$, $V_{0,y} = 0,45$, $B_z = 0,39$, $q = 1$.

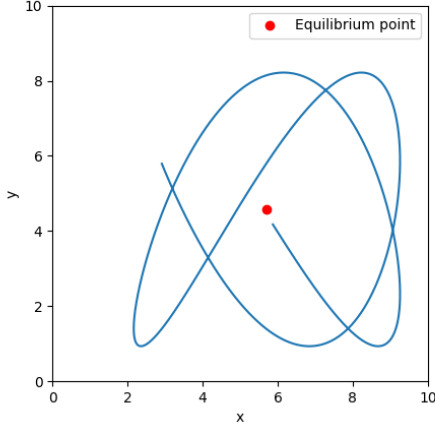


Figure 3: Example trajectory of a body moving due to harmonic oscillator force. The equilibrium position of the harmonic oscillator was also shown. Parameters: $x_0 = 5,87$, $y_0 = 4,17$, $V_{0,x} = 0,61$, $V_{0,y} = -0,96$, $r_{0,x} = 5,71$, $r_{0,y} = 4,58$, $k_x = 0,03$, $k_y = 0,07$.

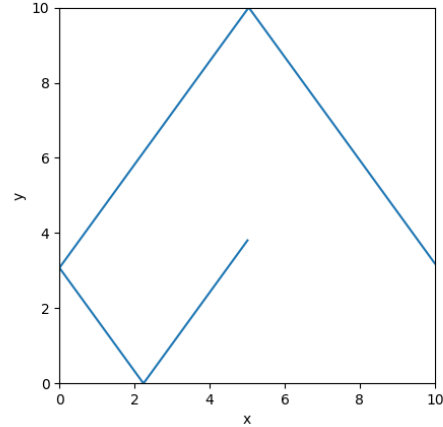


Figure 4: Example trajectory of a body moving freely (no force applied). In this case perfect elastic collisions of the body with walls were implemented. Parameters: $x_0 = 5,01$, $y_0 = 3,80$, $V_{0,x} = -0,56$, $V_{0,y} = -0,77$.

For the preliminary results, I used only gravitational simulations. I created 145 simulations with random starting positions, velocities, and directions of the gravity acceleration vector. Each simulation contained about 30 snapshots/frames. Such an example frame is shown in figure 5.

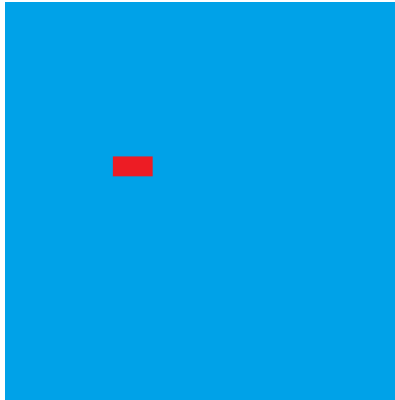


Figure 5: An example snapshot of one of the simulations of a body (the red rectangle) moving in a gravity field on the blue background.

2.2 Neural network architecture

The neural network architecture consists of autoencoder (encoder and decoder) and an evolution operator. The encoder consists of five convolutional ReLU layers (`torch.nn.Conv2d` and `torch.nn.ReLU` from PyTorch library) with kernel size 4 and padding 1, four of them with stride 2 followed by one with stride 1. At the end there is a fully connected linear layer (`torch.nn.Linear`) that gives a latent space vector of dimension 10. The number of channels goes from 3 for the input image to 32, 32, 64, 64 and 256 for the convolutional layers. The decoder network is a

mirror image of the encoder in terms of layer dimensions, with the convolution layers replaced by deconvolution layers (`torch.nn.ConvTranspose2d`). The evolution operator has three fully connected 32-neuron layers (`torch.nn.Linear`) with softplus activation function (`torch.nn.Softplus`) and a linear 10-neuron output layer. The evolution operator takes two vectors from latent space (the vector from the current and the previous time step) and predicts the latent space vector for the next time step. A graphical illustration of the architecture is shown in figure 6.

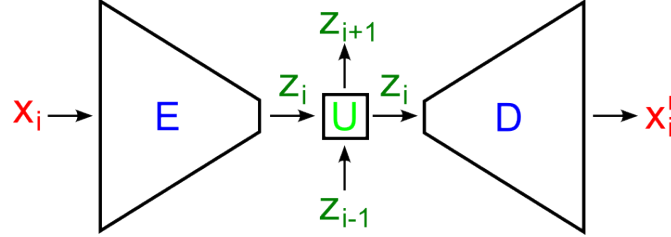


Figure 6: A draft of a neural network architecture consisting of encoder - E, decoder - D and an evolution operator - U. Encoder E encodes a snapshot from a video (a frame of a video with the moving body) - x_i into a low dimensional latent space vector z_i . The evolution operator U takes the latent space vector z_i and the latent space vector from a previous simulation time step - z_{i-1} (we receive it by encoding x_{i-1}), and returns the prediction of latent space vector of the following time step z_{i+1} . Then, the decoder D decodes the latent space vector z_i and returns the reconstructed snapshot from the video - x_i^r . In subsection 2.4 an implementation of the training loop of the network was presented. The symbols used on the figure correspond to the variables used in the code, in the following way: $x_i = x_curr$, $z_i = z_curr$, $z_{i-1} = z_prev$, $z_{i+1} = z_next_pred$, $x_i^r = reconstructed$ (in the code these variables appear for example in the line 57).

2.3 Loss function

The loss function [1] was implemented as follows:

$$\mathcal{L} = \mathcal{L}^{recon} + \alpha \mathcal{L}^{pred} + \beta \mathcal{L}^{nl} + \gamma \mathcal{L}^{acc}.$$

These four terms are averages over all time steps i of the following functions:

$$\begin{aligned} \mathcal{L}_i^{recon} &= \frac{|x_i - D(E(x_i))|}{|x_i|}, \\ \mathcal{L}_i^{pred} &= \frac{|z_{i+1} - U(z_i, z_{i-1})|}{|z_i - z_{i-1}|}, \\ \mathcal{L}_i^{nl} &= \frac{1}{4000} |z_i - z_{i-1}| \cdot \|\nabla \mathbf{J}(\mathbf{w}_i)\|_1, \\ \mathcal{L}_i^{acc} &= \frac{1}{10} \|U(\mathbf{w}_i) - \mathbf{M} \mathbf{w}_i\|_1, \end{aligned}$$

where α, β, γ are tunable hyperparameters, $\mathbf{w}_i = \begin{pmatrix} z_{i-1} \\ z_i \end{pmatrix}$, $\mathbf{J}$ is the Jacobian of an evolution operator U, $\mathbf{M} = \begin{pmatrix} -\mathbf{I} & 2\mathbf{I} \end{pmatrix}$ and $\mathbf{I}$ is the 10x10 identity matrix.

$\mathcal{L}^{recon}$ is the reconstruction error - it simply tells how good the encoder-decoder architecture preserves the input image, which we want to as good as possible. $\mathcal{L}^{pred}$ is the prediction error which tells how well the evolution operator U predicts the next latent space vector from two previous ones, which we also want to be as good as possible, so that the dynamics of the moving body is described well. $\mathcal{L}^{nl}$ is a measure of the nonlinearity of the mapping U , since its Jacobian $\mathbf{J}$ will be constant if the mapping is linear, i.e. the dynamics of the body is described by coupled linear differential equations, which encompass behaviour such as motion in magnetic fields, harmonic oscillator potential and gravity. $\mathcal{L}^{acc}$ is along with $\mathcal{L}^{nl}$ yet another measure of the nonlinearity of the mapping U and is a measure of the predicted acceleration. There is no acceleration if the mapping is $U(\mathbf{w}) = \mathbf{M}\mathbf{w}$.

2.4 Implementation of the training loop

In this subsection I presented the code implementation of the training loop. Code comments describe what each chunk of code does. Please take notice on the comment in the line 67, which describes the error made in the implementation of the training loop.

```

1  # Importing the necessary functions and classes from third party
   libraries (like torch) and from scripts I created myself (like
   autoencoder)
2  from torch import device, cuda, save
3  from torch.optim import Adam
4  from autoencoder import AutoencoderWithEvolution
5  from datasets import ImageDataset, collate_fn
6  from torch.utils.data import DataLoader
7  from torchvision import transforms
8  from evaluator import CombinedLoss
9  from tqdm import tqdm
10 import matplotlib.pyplot as plt
11
12 # Using GPU if available
13 device = device('cuda' if cuda.is_available() else 'cpu')
14 print('Device:', str(device))
15
16 # Initialising our autoencoder:
17 model = AutoencoderWithEvolution().to(device)
18
19 # Defining transformation of images:
20 transform = transforms.Compose([
21     transforms.Resize((64, 64)), # down sampling the resolution of the
   images
22     transforms.ToTensor()])
23
24 # Defining the class for calculating the loss function - criterion and
   an optimizer for training - optimizer
25 criterion = CombinedLoss(alpha=1.0, beta=1.0, gamma=1.0)
26 optimizer = Adam(model.parameters(), lr=1e-3)
27
28 num_epochs = 145 # 1 epoch is equivalent to training on one whole

```

```

simulation (approximately 30 frames)
29
30 # Defining tables that will store losses values after each epoch for
    latter plotting
31 total_Loss = []
32 total_reconstruction_Loss = []
33 total_prediction_error_Loss = []
34 total_non_linearity_Loss = []
35 total_acceleration_Loss = []
36
37 for epoch in range(num_epochs):
38
39     # Getting data out of one of the gravity simulations:
40     dataset =
        ImageDataset(directory=rf'simulation_frames/gravity_{epoch}/
                                simulation_snapshots',
                                transform=transform)
41
42     dataloader = DataLoader(dataset, batch_size=1, shuffle=False,
        collate_fn=collate_fn)
43
44     total_loss = 0
45     total_reconstruction_loss, total_prediction_error_loss,
        total_non_linearity_loss, total_acceleration_loss = (
46         0, 0 ,0 ,0)
47     for _, data in tqdm(dataloader):
48         # Getting snapshots out of one of the gravity simulations:
49         x_prev, x_curr, x_next = data.squeeze(0).to(device)
50         x_prev = x_prev.unsqueeze(0).requires_grad_(True)
51         x_curr = x_curr.unsqueeze(0).requires_grad_(True)
52         x_next = x_next.unsqueeze(0).requires_grad_(True)
53
54         optimizer.zero_grad()
55
56         # Encode and decode current frame using our model
57         reconstructed, z_curr, z_next_pred = model(x_curr,
            z_prev=model.encoder(x_prev))
58
59         # Encode next frame to get the ground truth z_next
60         z_next_true = model.encoder(x_next)
61
62         # Compute losses
63         reconstruction_loss, prediction_error_loss, non_linearity_loss,
            acceleration_loss, loss = criterion(
64             reconstructed, x_curr, x_next,
            z_prev=model.encoder(x_prev), z_curr=z_curr,
65             z_next_pred=z_next_pred, z_next_true=z_next_true,
            latent_dim=10)
66
67         # Backpropagation and optimization (this is erroneous as we
            should perform backpropagation after going through all time

```

```

        steps in one simulation - the following lines should be
        inside the outer loop)
68     loss.backward()
69     optimizer.step()
70
71     # Summing up losses
72     total_loss += loss.item()
73     total_reconstruction_loss += reconstruction_loss.item()
74     total_prediction_error_loss += prediction_error_loss.item()
75     total_non_linearity_loss += non_linearity_loss.item()
76     total_acceleration_loss += acceleration_loss.item()
77
78     # Printing losses after each epoch
79     print(f"Epoch {epoch + 1}, total loss: {total_loss /
80           len(dataloader):.4f}")
81     print(f"Epoch {epoch + 1}, reconstruction loss:
82           {total_reconstruction_loss / len(dataloader):.4f}")
83     print(f"Epoch {epoch + 1}, prediction error loss:
84           {total_prediction_error_loss / len(dataloader):.4f}")
85     print(f"Epoch {epoch + 1}, non linearity loss:
86           {total_non_linearity_loss / len(dataloader):.4f}")
87     print(f"Epoch {epoch + 1}, acceleration loss:
88           {total_acceleration_loss / len(dataloader):.4f}")
89
90     # Saving the calculated losses for latter plotting
91     total_Loss.append(total_loss / len(dataloader))
92     total_reconstruction_Loss.append(total_reconstruction_loss /
93           len(dataloader))
94     total_prediction_error_Loss.append(total_prediction_error_loss /
95           len(dataloader))
96     total_non_linearity_Loss.append(total_non_linearity_loss /
97           len(dataloader))
98     total_acceleration_Loss.append(total_acceleration_loss /
99           len(dataloader))
100
101     # Saving the model
102     save(model.state_dict(), r'saved_models/trained_on_10_simulations.pth')

```

3 Preliminary results

After performing a training session on all the simulations of a body moving in a gravity field I realised a big flaw in my training loop. Namely, the loss function was not calculated as described in subsection 2.3 as an average of all time steps in one simulation but after each frame. For in code details, see 2.4. Although the components of the loss function appeared to get smaller during training session, after visualizing the reconstructed images of snapshots of a simulation and comparing them with original images, it is clear that something is wrong. The graph of the values of loss function and its components with respect to the training epoch is presented in figure 7. The comparison between an original image and the reconstructed one is shown in figure 8.

I have also created an image of a reconstructed trajectory. By comparing it with the original trajectory, one could think it seems to be almost correct (neglecting some rotations and linear scaling). However, in my opinion, since I used flawed training loop, it is rather a coincidence. The comparison between an original trajectory with the reconstructed one is shown in figure 9.

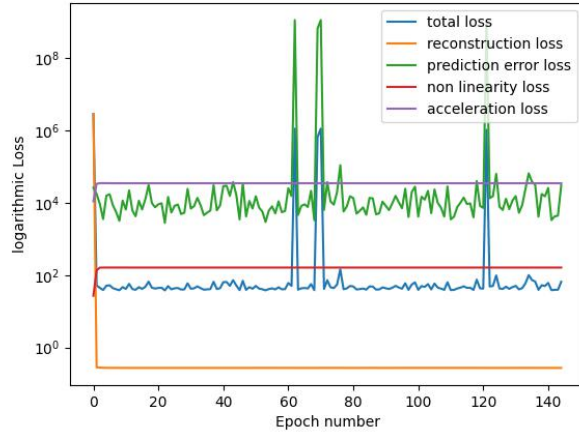


Figure 7: Components of the loss function versus epoch number during training. Each epoch corresponds to training performed on one simulation.

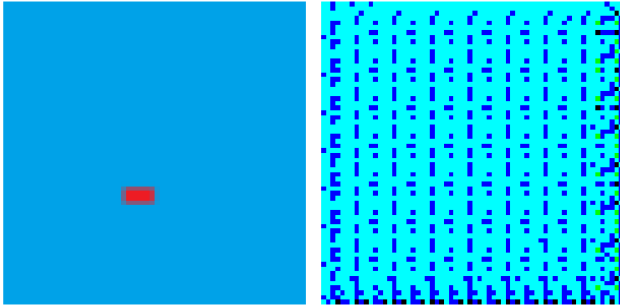


Figure 8: The comparison between original snapshot of a simulation (the photo is blurred as the images entering encoder are deliberately downsampled) on the left and the reconstructed image on the right. It is clear that reconstruction went wrong.

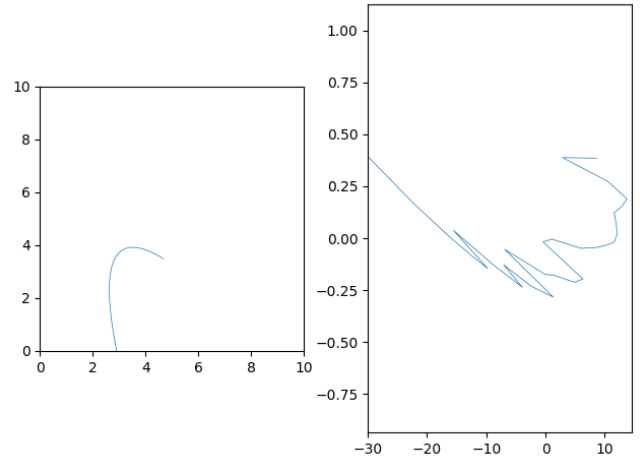


Figure 9: The comparison between an original trajectory of a body on the left with the reconstructed trajectory on the left.

4 Summary and future work

In the paper I presented the preliminary results of my attempt to reproduce the results of the article [1]. For now, I did not manage to obtain a correctly working algorithm that reconstructs

the trajectory of a moving object from a video due to a flawed training loop procedure.

In the future work, I will concentrate on repairing the issue with training loop and try to obtain the desired results. The work that is still ahead is implementing a comprehensive evaluator that would assess the quality of latent space trajectory representations and extending the training of the model on other types of motion like movement in magnetic field, harmonic oscillator. In the end it would be interesting to see, if the neural network architecture would be able to detect the trajectories of bodies from real footages.

References

- [1] Silviu-Marian Udrescu and Max Tegmark. “Symbolic pregression: Discovering physical laws from distorted video”. In: *Phys. Rev. E* 103 (4 Apr. 2021), p. 043307. DOI: 10.1103/PhysRevE.103.043307. URL: <https://link.aps.org/doi/10.1103/PhysRevE.103.043307>.
- [2] Silviu-Marian Udrescu and Max Tegmark. *AI Feynman: a Physics-Inspired Method for Symbolic Regression*. 2020. arXiv: 1905.11481 [physics.comp-ph]. URL: <https://arxiv.org/abs/1905.11481>.
- [3] Alexandre Koyré. *The Astronomical Revolution: Copernicus-Kepler-Borelli*. Routledge, 2013.